LAB5

Data Structure

Lab5

- The deadline for lab5 submission is 4/2 (THU) at 11:59 pm.
- If you have any questions, please contact TA through email. (112alsdud@naver.com)
- Lab scores will be uploaded to the LMS next week.
- Questions about Lab 5 scores will only be accepted during next lab session.

- Code name: {student_id}_p5.c
- The code will be tested with 5 different input files.
- Each input file will be worth 20 points
- Make sure that the output format matches the provided example exactly.

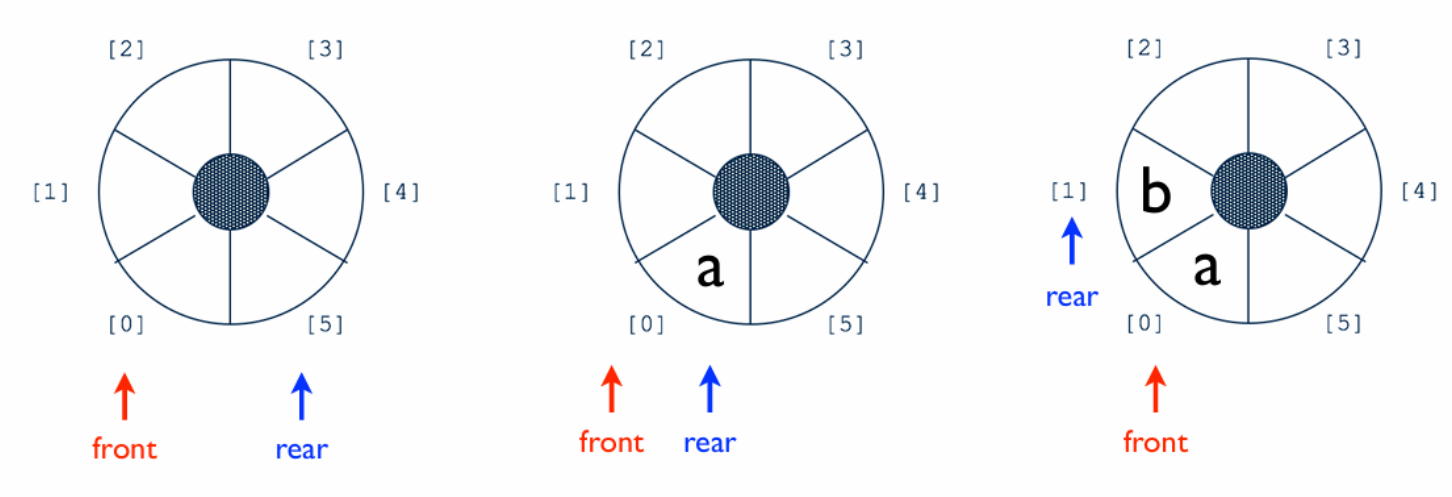
Evaluation criteria

Category	Evaluation	
p5	100	
Total	100	

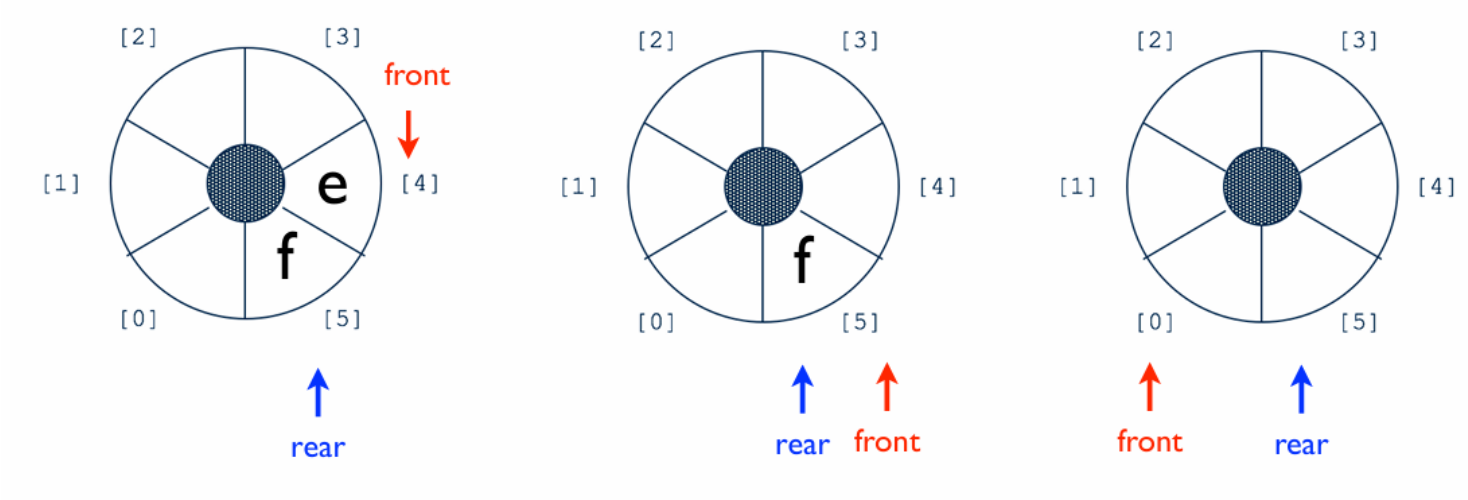
- *Use GCC **13.3** version.*
- *No score will be given if there is a compile error caused by difference of gcc version*

Lab5 – CircularQueue

- **Enqueue**



- **Dequeue**



Lab5 –Ticketing System Using CircularQueue

- Implement a circular queue using dynamic memory allocation to simulate a ticketing system
- The input file contains commands followed by necessary arguments:
  - **c x** → create a new queue with the size of x. The number x is the maximum size of the queue. All the input files will start with "c x"
  - **e [Name]** → Enqueue : Register a customer and assign ticket number
  - **d** → Dequeue : Serve the first customer
  - **p** → Print : Show the current queue

```
c 3
e Alice
e Bob
e Charlie
p
d
e Diana
p
d
d
d
p
```

Ticketing System with CircularQueue

```
Ticket issued: #1 for Alice
Ticket issued: #2 for Bob
Ticket issued: #3 for Charlie

Waiting list:
   #1 - Alice
   #2 - Bob
   #3 - Charlie

Now serving: #1 - Alice
Ticket issued: #4 for Diana

Waiting list:
   #2 - Bob
   #3 - Charlie
   #4 - Diana

Now serving: #2 - Bob
Now serving: #3 - Charlie
Now serving: #4 - Diana

Waiting list:
```

Lab5 –Ticketing System Using CircularQueue

- **Your program should include:**
  - CircularQueue MakeEmpty(int maxSize);
  - int IsEmpty(CircularQueue q);
  - int IsFull(CircularQueue q);
  - void enqueue(CircularQueue q, char* name);
  - void dequeue(CircularQueue q);
  - void printQueue(CircularQueue q);
  - void freeQueue(CircularQueue q);

Lab5 –Ticketing System Using CircularQueue

- **CircularQueue MakeEmpty(int maxSize)**
  - Create a new circular queue with maxSize.
- **int IsEmpty(CircularQueue q)**
  - Check whether the queue is empty or not.
- **int IsFull(CircularQueue q)**
  - Check whether the queue is full or not.
- **void enqueue(CircularQueue q, char* name)**
  - Add a new customer with the given name to the queue and assigns a unique ticket number.
  - On success, print a message "**Ticket issued: #[ticketNumber] for [name]**".
  - If your queue is full, just print an error message "**The waiting queue is full.**" and pass that customer.

Lab5 –Ticketing System Using CircularQueue

- **void dequeue(CircularQueue q)**
  - Serves and removes the front customer.
  - On success, print a message "**Now serving: #[ticketNumber] – [name]**".
  - If your queue is empty, just print an error message "**No one is waiting.**".
- **void printQueue(CircularQueue q)**
  - Displays all customers in queue from front to rear in a formatted way.

    "**Waiting list:**

    #[ticketNumber1] – [name1]

    #[ticketNumber2] – [name2]"
- **void freeQueue(CircularQueue q)**
  - Delete the queue and free all the memory allocated to queue.

Lab5 –Ticketing System Using CircularQueue

- Structure - Customer

```
typedef struct CustomerStruct{
    int ticketNumber;
    char name[NAME_LEN];
} Customer;
```

- Structure - CircularQueue

```
typedef struct CircularQueueStruct {
    Customer* queue;
    int front;
    int rear;
    int count;
    int maxSize;
    int nextTicket;
} *CircularQueue;
```

- Function

```
CircularQueue MakeEmpty(int maxSize);
int IsEmpty(CircularQueue q);
int IsFull(CircularQueue q);
void enqueue(CircularQueue q, char* name);
void dequeue(CircularQueue q);
void printQueue(CircularQueue q);
void freeQueue(CircularQueue q);
```

Lab5 –Ticketing System Using CircularQueue

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define NAME_LEN 50

typedef struct CustomerStruct{
    int ticketNumber;
    char name[NAME_LEN];
} Customer;

typedef struct CircularQueueStruct {
    Customer* queue;
    int front;
    int rear;
    int count;
    int maxSize;
    int nextTicket;
} *CircularQueue;

CircularQueue MakeEmpty(int maxSize);
int IsEmpty(CircularQueue q);
int IsFull(CircularQueue q);
void enqueue(CircularQueue q, char* name);
void dequeue(CircularQueue q);
void printQueue(CircularQueue q);
void freeQueue(CircularQueue q);
```

```c
int main(int argc, char* argv[]) {
    FILE* file = fopen(argv[1], "r");
    if (!file) {
        printf("Failed to open input.txt\n");
        return 1;
    }

    char command;
    char name[NAME_LEN];
    int size;
    CircularQueue q;

    while (fscanf(file, " %c", &command) != EOF) {
        switch (command) {
            case 'c':
                fscanf(file, " %d", &size);
                q = MakeEmpty(size);
                break;
            case 'e':
                fscanf(file, " %[^\n]", name);
                enqueue(q, name);
                break;
            case 'd':
                dequeue(q);
                break;
            case 'p':
                printQueue(q);
                break;
            default:
                printf("Unknown command!\n");
        }
    }

    fclose(file);
    freeQueue(q);
    return 0;
}
```

Lab5 –Ticketing System Using CircularQueue

- input file : input.txt

```
c 3
e Alice
e Bob
e Charlie
e Jhon
p
d
e Diana
d
p
d
d
d
p
```

- Result

```
Ticket issued: #1 for Alice
Ticket issued: #2 for Bob
Ticket issued: #3 for Charlie
The waiting queue is full.

Waiting list:
   #1 - Alice
   #2 - Bob
   #3 - Charlie

Now serving: #1 - Alice
Ticket issued: #4 for Diana
Now serving: #2 - Bob

Waiting list:
   #3 - Charlie
   #4 - Diana

Now serving: #3 - Charlie
Now serving: #4 - Diana
No one is waiting.

Waiting list:
```